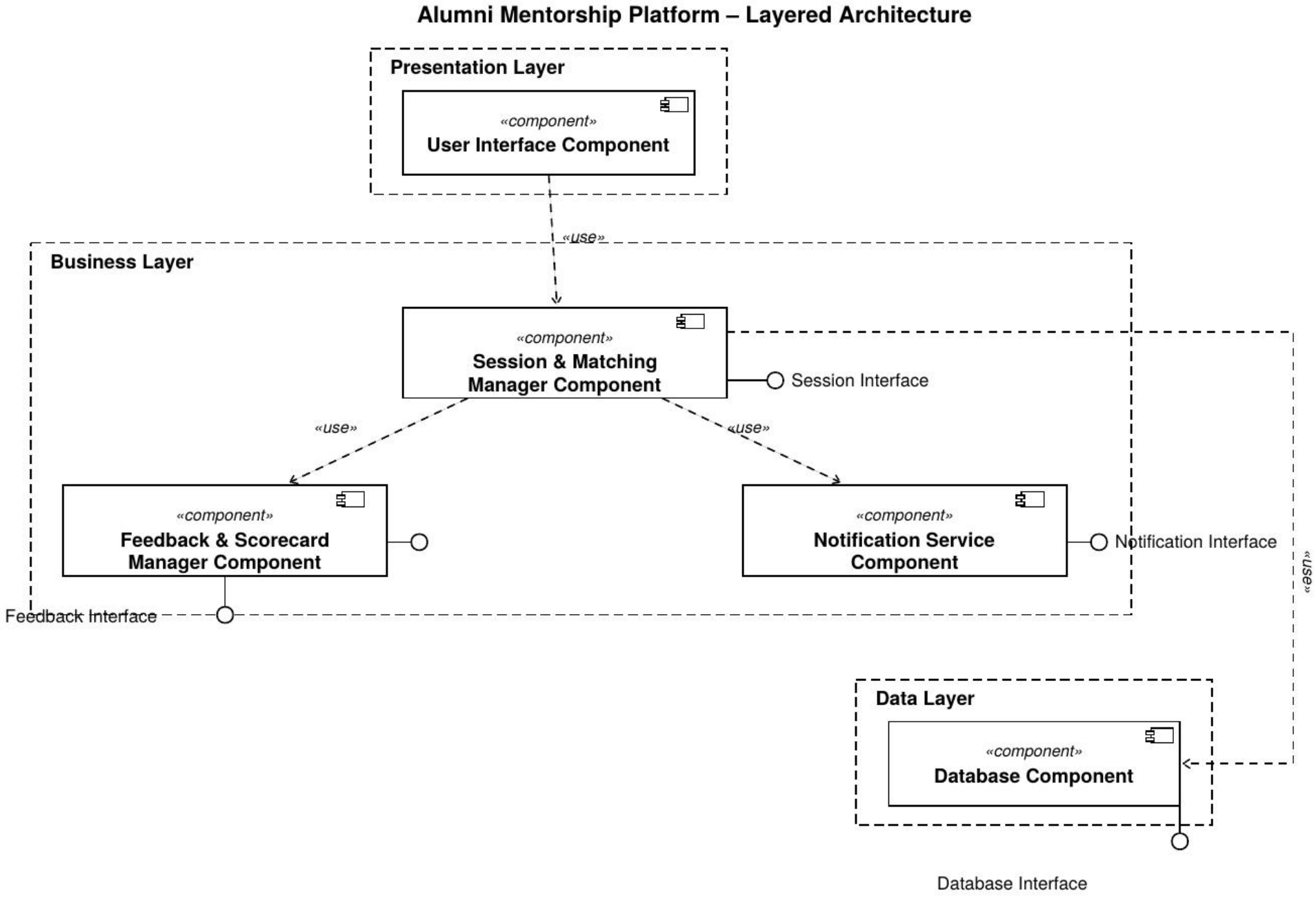


Component diagram for the Alumni Mentorship & Mock Interview Platform, following a three-layer architecture.

Lab 3: Architecture Selection & Justification

Project: Alumni Mentorship & Mock Interview Platform

We chose Layered Architecture for the Alumni Mentorship & Mock Interview Platform System.

Architectural Choice

We selected a Layered Architecture, organizing the system into a Presentation Layer (User Interface Component), a Business Layer (Session & Matching Manager, Feedback & Scorecard Manager, and Notification Service components), and a Data Layer (Database Component).

Reasons for This Choice

1. Clear separation of concerns for a single deployable application: The platform's core workflows (mentor matching, booking, feedback, and notifications) are business rules that should be isolated from the UI and from data storage. A layered structure keeps the UI free of matching/scheduling logic and keeps the database access isolated behind the business layer, making the system easier to test, maintain, and extend (e.g., adding a new interview-scheduling rule) without touching the UI or storage code.

2. Matches the project's scale and deployment model: This is a moderate-scale platform expected to serve a bounded population of students and alumni mentors, not an internet-scale system needing independently scaled services. Microservices would introduce unnecessary operational complexity (service discovery, distributed transactions) for functionality that is naturally cohesive, while Client-Server alone would not adequately separate scheduling, feedback, and notification concerns.

Security Advantage

Because the Database Component is only accessible through the Business Layer, all requests for profile data, session records, and scorecards pass through centralized access-control and validation logic before reaching storage. This makes it straightforward to enforce the platform's data-privacy requirement (soft-deleting user data within 24 hours of a verified request) consistently in one place, rather than duplicating privacy checks across multiple independent services.

Performance Benefit

Keeping all business logic within a single layer allows the Session & Matching Manager to query the database directly and cache frequently accessed data (e.g., ranked mentor-search results) in-process, avoiding the network hops and serialization overhead that a distributed microservices call chain would introduce. This directly supports the requirement that domain-matching search results return within 3 seconds for 95% of requests under up to 500 concurrent users.